

■# Documentation System

Platform Version: 0.0.4

Author: Meow Li

Copyright: Copyright (c) 2026 Meow Li. All Rights Reserved.

This document explains how the project documentation is organized. It is the first file a new human contributor or AI agent should read when entering the repository from a fresh start.

Documentation Goals

The documentation has four jobs:

1. Explain the product clearly to bridge players, potential users, and potential investors.
2. Teach users how to use the platform, from basic system selection to advanced custom gadget authoring.
3. Give engineers and AI agents enough technical detail to recover the project logic from code and docs alone.
4. Preserve open questions and future product ideas without mixing them into active specs.

Current Documents

PDF copies of active Markdown documents are generated under ``docs/pdf/``. The Markdown files are the editable source of truth; PDFs are distribution artifacts for reading and sharing. Between releases, PDF files may intentionally lag behind Markdown edits because PDFs are regenerated only for releases or explicit PDF-generation requests.

Product-Facing Documents

`docs/product\_description.md`

Audience:

- ? Bridge players.
- ? Teachers and partnership coaches.
- ? Club or tournament organizers.
- ? Potential investors or business partners.
- ? Users who can use web tools but do not need programming knowledge.

Purpose:

- ? Describe what the platform is.
- ? Explain the human-plus-custom-bot game model.
- ? Explain why strict agreement execution matters.
- ? Present the business/product opportunity.
- ? Give examples of likely user workflows.

This document should avoid implementation details except where they help explain the product.

``docs/user_manual.md``

Audience:

- ? New users learning the platform.
- ? Intermediate bridge players configuring a system.
- ? Advanced players writing custom gadgets.
- ? Power users testing bidding methods.

Purpose:

- ? Beginner guide.
- ? System configuration guide.
- ? Explanation-output guide.
- ? Advanced gadget-authoring guide.

This document should stay clean and practical. Planned or missing work belongs in ``docs/todo.md``, not in the user manual.

Engineering And AI Recovery Documents

``docs/implementation_design.md``

Audience:

- ? Software engineers.
- ? AI coding agents.
- ? Technical project maintainers.

Purpose:

- ? File-by-file implementation guide.
- ? Class, field, function, and variable-level explanation.
- ? Current schemas and output contracts.
- ? Known technical limitations.

This is the primary code-reading guide. A fresh AI agent should be able to inspect the code line by line with this document open.

``docs/semantic_ontology.md``

Audience:

- ? Software engineers.
- ? AI coding agents.
- ? Advanced gadget authors.

? Future LLM-gadget-generation tooling.

Purpose:

? Define the formal bridge-domain vocabulary shared by gadgets and engine code.

? Specify executable expectations for semantic concepts such as transfers, agreed suits, forcing status, competitive state, and keycard context.

? Prevent portable gadgets from depending on arbitrary private meaning strings.

This document is the system-level semantic contract. The implementation document explains the current code; the ontology document explains the target vocabulary the code should implement.

`docs/bridge\_system\_language\_roadmap.md`

Audience:

? Product owner.

? Software engineers.

? AI coding agents.

? Future GUI and compiler implementers.

Purpose:

? Define the roadmap from direct YAML authoring toward Bridge System Language, compiled intermediate representation, and GUI authoring tools.

? Clarify that BSL is a formal authoring language, while the engine executes validated IR.

? Preserve the design rule that forms, BSL, and IR/YAML should all be available for different users and tasks.

`docs/todo.md`

Audience:

? Product owner.

? Engineers.

? AI agents.

Purpose:

? Running list of future ideas, open problems, and user notes.

? Product thoughts that should not be lost in chat history.

When the user says "put this in the TODO list," update this file.

Documentation Authority Order

When documents conflict, use this order:

1. Current code and tests.
2. `docs/semantic\_ontology.md` for formal bridge-domain semantic vocabulary.
3. `docs/bridge\_system\_language\_roadmap.md` for authoring-language and GUI roadmap decisions.
4. `docs/implementation\_design.md` for technical implementation.
5. `docs/documentation\_system.md` for doc organization.
6. `docs/product\_description.md` for product framing.
7. `docs/user\_manual.md` for user-facing behavior and examples.
8. `docs/todo.md` for planned but unresolved items.

Terminology Policy

Use current formal terms:

- ? Use **gadget** for a portable convention module.
- ? Use **rule** for the formal bidding logic object.
- ? Use **suit** for the bidding suit field `C D H S N`.
- ? Use `N` for notrump in canonical calls.
- ? Use `P`, `X`, and `R` for pass, double, and redouble.
- ? Use lowercase `n e s w` for seats.
- ? Use **basic system** or **base system** only as user-facing language. In backend implementation it is still a normal gadget.
- ? Store each gadget in its own directory with `gadget.yaml` metadata plus one or more rule YAML files.

Maintenance Rules

1. Any code change must update `docs/implementation\_design.md` if it changes structure, behavior, schemas, output, or limitations.
2. Any product-level change should update `docs/product\_description.md`.
3. Any user workflow or gadget-authoring change should update `docs/user\_manual.md`.
4. Any new open question, future feature, or deferred idea should update `docs/todo.md`.
5. Any new design instruction from the user must be reflected in the appropriate active documents in the same checkpoint. Do not leave product logic, terminology decisions, or architecture rules only in chat history.
6. Route new design information by audience: product framing to `product\_description.md`, user-facing workflows to `user\_manual.md`, technical behavior to `implementation\_design.md`, and planned/deferred work to `todo.md`.
7. Any change to documentation organization should update this file.
8. For each platform version update of the code, the full active documentation system must be reviewed and updated together. This includes version metadata, current behavior, examples, and limitations.
9. PDF files are release artifacts. During intermediate editing, update the Markdown source documents only and do not regenerate PDFs.
10. Regenerate PDF copies only when the user explicitly says to release a new version, or when the user separately asks for PDF regeneration.
11. The version number in active documents is the platform version, not an individual gadget version. Each new platform version should update the version metadata in every active Markdown document and in the generated PDFs.

12. The platform version number changes only when the user explicitly says to release a new version. Routine edits, TODO additions, documentation improvements, and code checkpoints should keep the existing platform version unless the user asks for a release.

13. When the user says "make a new version," "release a new version," or equivalent release language, the agent should update the platform version, review and update active documentation, regenerate PDF artifacts, run the test suite, and publish/upload the checkpoint to the configured GitHub `Partner` project when GitHub access is available.

Fresh-Start Recovery Path

A new AI agent or engineer should read in this order:

1. `docs/documentation\_system.md`
2. `docs/product\_description.md`
3. `docs/user\_manual.md`
4. `docs/semantic\_ontology.md`
5. `docs/bridge\_system\_language\_roadmap.md`
6. `docs/implementation\_design.md`
7. `docs/todo.md`
8. Relevant code files referenced by `implementation\_design.md`

After reading those files, the agent should be able to answer:

- ? What product are we building?
- ? Who is it for?
- ? What is a gadget?
- ? How are calls represented?
- ? How does the engine replay an auction?
- ? How does it separate public meaning from internal origin?
- ? What formal semantic vocabulary do gadgets share?
- ? How should BSL, IR/YAML, GUI forms, and the engine relate?
- ? What is implemented now?
- ? What planned work is tracked in `docs/todo.md`?